
TransitOps

Phase 1 — Technical Implementation Report

Smart Transport Operations Platform · Hackathon Build

Phase	Date	Stack	Status
Phase 1 of 5	July 12, 2026	Python · Streamlit · SQLAlchemy · SQLite	Complete

[1] Overview

Phase 1 establishes the complete foundation for TransitOps: project structure, database models, authentication, role-based access control, dark-themed UI, and a fully working dashboard. All subsequent phases build on this without structural changes.

- Project scaffold with layered architecture (app/ backend, frontend/ UI)
- .env + app/config.py — centralized settings, no magic strings
- 8 SQLAlchemy ORM models (Role, User, Vehicle, Driver, Trip, MaintenanceLog, FuelLog, Expense)
- bcrypt password hashing + session-based authentication
- RBAC permission matrix — 4 roles × 8 modules × 3 access levels
- Auto-seeded demo data: 4 roles, 4 users, 6 vehicles, 5 drivers, 5 trips
- Dark theme CSS + Streamlit config.toml with orange accent
- Login page with 4 quick-fill role buttons
- Dashboard with 7 live KPI tiles + recent trips table + vehicle status chart
- 8 navigable pages (all functional, Phases 2–5 implement full CRUD)
- Status enums (constants.py), structured logger, frontend router module

[2] Architecture Decision — Single Entry Point

Decision: The application uses a single app.py entry point instead of Streamlit's built-in multipage routing system.

Why Streamlit's native multipage was dropped:

- Pages must live in a root-level pages/ directory — the SDD's frontend/pages/ layout is unsupported
- st.switch_page() rejects paths like frontend/pages/1_Dashboard.py
- RBAC-filtered navigation (hiding pages per role) conflicts with Streamlit's auto-generated sidebar nav
- Each page re-runs independently, making centralized auth guard and CSS loading fragile

Solution — render() Pattern:

Each page module in frontend/pages/ exposes a single render() function. app.py manages auth, CSS, sidebar, and dispatches to the correct render() via frontend/router.py.

Aspect	Streamlit Multipage	render() Pattern (chosen)
Pages location	pages/ at root only	Anywhere (module import)
RBAC	Manual per-page	Centralized in router
Auth guard	Manual per-page	Centralized in app.py

CSS loading	Manual per-page	Once in app.py
SDD compliance	Breaks structure	Fully preserves SDD layout

[3] Issues Encountered & Fixes (smallest to biggest)

Bug 1 — Windows cp1252 Emoji Encoding Error [Minor]

`UnicodeEncodeError: charmap codec cannot encode emoji characters`

seed.py and engine.py used emoji (🔴🔵🟢) inside print() calls. Windows terminal defaults to cp1252, which cannot encode UTF-8 emoji codepoints.

Fix: Replaced all emoji in print() calls with ASCII text. Later replaced all print() with the app.logger module which handles encoding safely.

Bug 2 — Streamlit Auto-Nav Conflict [Cosmetic]

When pages were temporarily in the pages/ directory, Streamlit auto-generated its own sidebar navigation on top of our custom sidebar, producing a duplicate navigation bar.

Fix: Added CSS to style.css to suppress Streamlit's auto-generated nav element.

```
[data-testid="stSidebarNav"] { display: none !important; }
```

Bug 3 — Streamlit Multipage Routing Failure [Critical — Architecture Change]

`StreamlitAPIException: Could not find page: frontend/pages/1_Dashboard.py. Only the main app file and files in the pages/ directory are supported.`

This fired on login when st.switch_page("frontend/pages/1_Dashboard.py") was called. Streamlit's multipage system only accepts paths inside a root-level pages/ directory, making our SDD-compliant folder structure impossible to use.

Fix: Completely replaced Streamlit's multipage routing with the single-entry render() pattern described in Section 2. Pages moved back to frontend/pages/ (SDD-compliant), renamed from 1_Dashboard.py → page_dashboard.py, each file exposes render() instead of running as a standalone Streamlit script.

[4] Features Implemented

4.1 Database Models (8 Tables)

Model	Table	Key Fields	Purpose
Role	roles	name, permissions_json	4 system roles
User	users	email, password_hash, role_id	Login credentials

Vehicle	vehicles	registration_no, status, max_capacity_kg	Fleet asset registry
Driver	drivers	license_no, license_expiry, safety_score	Driver profiles
Trip	trips	trip_code, status, vehicle_id, driver_id	Core operational entity
MaintenanceLog	maintenance_logs	vehicle_id, service_type, status	Service records
FuelLog	fuel_logs	vehicle_id, liters, cost	Fuel consumption
Expense	expenses	vehicle_id, category, amount	Tolls & misc costs

4.2 RBAC Permission Matrix

Module	Fleet Manager	Dispatcher	Safety Officer	Financial Analyst
Dashboard	view	view	view	view
Fleet	edit	view	view	view
Drivers	edit	view	edit	view
Trips	view	edit	view	view
Maintenance	edit	view	view	view
Fuel & Expenses	edit	view	view	edit
Analytics	view	view	view	view
Settings	edit	none	none	none

4.3 Seed Data

Entity	Count	Status Values Covered
Roles	4	Fleet Manager, Dispatcher, Safety Officer, Financial Analyst
Users	4	One per role
Vehicles	6	Available ×3, On Trip ×1, In Shop ×1, Retired ×1
Drivers	5	Available ×2, On Trip ×1, Off Duty ×1, Suspended ×1
Trips	5	Completed ×2, Dispatched ×1, Draft ×1, Cancelled ×1

4.4 Dashboard (Screen 1)

- 7 KPI tiles: Total Vehicles, Available, In Maintenance, Active Trips, Pending Trips, Drivers On Duty, Fleet Utilization %

- Filters row: Vehicle Type, Status, Region (UI ready — wired to data in Phase 2)
- Recent Trips table: Last 5 trips with color-coded status column
- Vehicle Status chart: Horizontal bar chart via Plotly, dark background

4.5 Placeholder Pages (Screens 2–8)

Page	Phase 1 Content	Full Implementation
Fleet	Read-only vehicle table	Phase 2
Drivers	Driver table + license expiry flag	Phase 2
Trips	Read-only trips table	Phase 3
Maintenance	In Shop count metric	Phase 4
Fuel & Expenses	Cost totals (fuel + expenses)	Phase 4
Analytics	Fuel efficiency, utilization, revenue metrics	Phase 5
Settings	Read-only RBAC matrix (Fleet Manager only)	Phase 5

[5] Post-Phase Enhancements

app/constants.py — Status Enums

Replaces magic strings like "Available", "On Trip", "Draft" across models, seed data, and UI color maps. str-based enums are drop-in compatible with SQLAlchemy, pandas, and Streamlit without casting. 6 enums: VehicleStatus, DriverStatus, TripStatus, MaintenanceStatus, UserRole, VehicleType.

app/logger.py — Centralized Logger

Replaces bare print() calls in the database layer. get_logger(__name__) factory returns a named logger with consistent [HH:MM:SS] LEVEL module: msg format. Log level is driven by the DEBUG flag in .env.

frontend/router.py — Page Dispatcher

Extracted route_to_page() and PAGE_MAP dict out of app.py. Adding a new page in Phase 2+ requires only one new line in PAGE_MAP. RBAC is double-checked here as well.

Login Quick-Fill Buttons

4 role buttons on the login page (Fleet Mgr, Dispatcher, Safety, Finance). Clicking sets st.session_state for email and password inputs and reruns — the form renders pre-filled instantly.

[6] Demo Credentials

Role	Email	Password	Access Highlights
Fleet Manager	fleet@transitops.com	fleet123	All 8 pages incl. Settings
Dispatcher	dispatch@transitops.com	dispatch123	Dashboard, Trips (edit)
Safety Officer	safety@transitops.com	safety123	Dashboard, Drivers (edit)
Financial Analyst	finance@transitops.com	finance123	Dashboard, Fuel & Expenses (edit)

[7] Smoke Test Results

```
[OK] config APP_NAME=TransitOps, DEBUG=True
[OK] models all 8 imported cleanly
[OK] hashing bcrypt hash+verify working
[OK] rbac 4-role matrix correct, FA cannot see Settings
[OK] database init_db + seed_all with structured logger
[OK] seed {roles:4, users:4, vehicles:6, drivers:5, trips:5}
[OK] auth all 4 login credentials verified
[OK] pages all 8 files exist with render()
[OK] app.py uses router.py and logger
[OK] constants enums + color maps
[OK] logger [HH:MM:SS] format working
```

[8] Developer Commands

Command	Effect
<code>streamlit run app.py</code>	Start app (DB auto-inits + seeds on first run)
<code>python -c "from app.database.engine import reset_database; reset_database()"</code>	Nuke DB, recreate tables, reseed all demo data
<code>pip install -r requirements.txt</code>	Install all dependencies

[9] Phase 2 Preview

- Vehicle Registry CRUD: Add Vehicle form with Pydantic validation, Edit Vehicle, status management
- Driver Management CRUD: Add Driver form, Edit Driver, license expiry alerts
- Pydantic Schemas: VehicleCreate, DriverCreate with business rule validation
- Service layer: vehicle_service.py, driver_service.py — all DB logic stays out of UI

- Search & Filter: filter Fleet/Driver tables by type, status, region